

```

void remdiv(long x, long y, long *qp, long *rp)
x in %rdi, y in %rsi, qp in %rdx, rp in %rcx
1  remdiv:
2      movq    %rdx, %r8      Copy qp
3      movq    %rdi, %rax     Move x to lower 8 bytes of dividend
4      cqto     Sign-extend to upper 8 bytes of dividend
5      idivq    %rsi          Divide by y
6      movq    %rax, (%r8)    Store quotient at qp
7      movq    %rdx, (%rcx)   Store remainder at rp
8      ret

```

在上述代码中，必须首先把参数 `qp` 保存到另一个寄存器中(第2行)，因为除法操作要使用参数寄存器 `%rdx`。接下来，第3~4行准备被除数，复制并符号扩展 `x`。除法之后，寄存器 `%rax` 中的商被保存在 `qp`(第6行)，而寄存器 `%rdx` 中的余数被保存在 `rp`(第7行)。

无符号除法使用 `divq` 指令。通常，寄存器 `%rdx` 会事先设置为0。

 **练习题 3.12** 考虑如下函数，它计算两个无符号 64 位数的商和余数：

```

void uremdiv(unsigned long x, unsigned long y,
             unsigned long *qp, unsigned long *rp) {
    unsigned long q = x/y;
    unsigned long r = x%y;
    *qp = q;
    *rp = r;
}

```

修改有符号除法的汇编代码来实现这个函数。

3.6 控制

到目前为止，我们只考虑了直线代码的行为，也就是指令一条接一条顺序地执行。C语言中的某些结构，比如条件语句、循环语句和分支语句，要求有条件的执行，根据数据测试的结果来决定操作执行的顺序。机器代码提供两种基本的低级机制来实现有条件的行为：测试数据值，然后根据测试的结果来改变控制流或者数据流。

与数据相关的控制流是实现有条件行为的更一般和更常见的方法，所以我们先来介绍它。通常，C语言中的语句和机器代码中的指令都是按照它们在程序中出现的次序，顺序执行的。用 `jump` 指令可以改变一组机器代码指令的执行顺序，`jump` 指令指定控制应该被传递到程序的某个其他部分，可能是依赖于某个测试的结果。编译器必须产生构建在这种低级机制基础之上的指令序列，来实现C语言的控制结构。

本文会先涉及实现条件操作的两种方式，然后描述表达循环和 `switch` 语句的方法。

3.6.1 条件码

除了整数寄存器，CPU还维护着一组单个位的条件码(condition code)寄存器，它们描述了最近的算术或逻辑操作的属性。可以检测这些寄存器来执行条件分支指令。最常用的条件码有：

CF：进位标志。最近的操作使最高位产生了进位。可用来检查无符号操作的溢出。

ZF：零标志。最近的操作得出的结果为0。

SF：符号标志。最近的操作得到的结果为负数。

OF：溢出标志。最近的操作导致一个补码溢出——正溢出或负溢出。

比如说，假设我们用一条 ADD 指令完成等价于 C 表达式 $t=a+b$ 的功能，这里变量 a 、 b 和 t 都是整型的。然后，根据下面的 C 表达式来设置条件码：

CF	(unsigned) $t < (\text{unsigned}) a$	无符号溢出
ZF	$(t == 0)$	零
SF	$(t < 0)$	负数
OF	$(a < 0 == b < 0) \ \&\& \ (t < 0 != a < 0)$	有符号溢出

leaq 指令不改变任何条件码，因为它是用来进行地址计算的。除此之外，图 3-10 中列出的所有指令都会设置条件码。对于逻辑操作，例如 XOR，进位标志和溢出标志会设置成 0。对于移位操作，进位标志将设置为最后一个被移出的位，而溢出标志设置为 0。INC 和 DEC 指令会设置溢出和零标志，但是不会改变进位标志，至于原因，我们就不在这里深入探讨了。

除了图 3-10 中的指令会设置条件码，还有两类指令（有 8、16、32 和 64 位形式），它们只设置条件码而不改变任何其他寄存器；如图 3-13 所示。CMP 指令根据两个操作数之差来设置条件码。除了只设置条件码而不更新目的寄存器之外，CMP 指令与 SUB 指令的行为是一样的。在 ATT 格式中，列出操作数的顺序是相反的，这使代码有点难读。如果两个操作数相等，这些指令会将零标志设置为 1，而其他的标志可以用来确定两个操作数之间的大小关系。TEST 指令的行为与 AND 指令一样，除了它们只设置条件码而不改变目的寄存器的值。

指令	基于	描述
CMP S_1, S_2	$S_2 - S_1$	比较
cmpb		比较字节
cmpw		比较字
cmpl		比较双字
cmpq		比较四字
TEST S_1, S_2	$S_1 \& S_2$	测试
testb		测试字节
testw		测试字
testl		测试双字
testq		测试四字

图 3-13 比较和测试指令。这些指令不修改任何寄存器的值，只设置条件码

典型的用法是，两个操作数是一样的（例如，`testq %rax,%rax` 用来检查 `%rax` 是负数、零，还是正数），或其中的一个操作数是一个掩码，用来指示哪些位应该被测试。

3.6.2 访问条件码

条件码通常不会直接读取，常用的使用方法有三种：1) 可以根据条件码的某种组合，将一个字节设置为 0 或者 1，2) 可以条件跳转到程序的某个其他的部分，3) 可以有条件地传送数据。对于第一种情况，图 3-14 中描述的指令根据条件码的某种组合，将一个字节设置为 0 或者 1。我们将这一整类指令称为 SET 指令；它们之间的区别就在于它们考虑的条件码的组合是什么，这些指令名字的不同后缀指明了它们所考虑的条件码的组合。这些指令的后缀表示不同的条件而不是操作数大小，了解这一点很重要。例如，指令 `setl` 和 `setb` 表示“小于时设置(set less)”和“低于时设置(set below)”，而不是“设置长字(set long word)”和“设置字节(set byte)”。

一条 SET 指令的目的操作数是低位单字节寄存器元素(图 3-2)之一，或是一个字节的内存位置，指令会将这个字节设置成 0 或者 1。为了得到一个 32 位或 64 位结果，我们必须对高位清零。一个计算 C 语言表达式 $a < b$ 的典型指令序列如下所示，这里 a 和 b 都是 long 类型：

指令	同义名	效果	设置条件
sete <i>D</i>	setz	$D \leftarrow ZF$	相等/零
setne <i>D</i>	setnz	$D \leftarrow \neg ZF$	不等/非零
sets <i>D</i>		$D \leftarrow SF$	负数
setns <i>D</i>		$D \leftarrow \neg SF$	非负数
setg <i>D</i>	setnle	$D \leftarrow \neg(SF \wedge OF) \wedge \neg ZF$	大于(有符号>)
setge <i>D</i>	setnl	$D \leftarrow \neg(SF \wedge OF)$	大于等于(有符号>=)
setl <i>D</i>	setnge	$D \leftarrow SF \wedge OF$	小于(有符号<)
setle <i>D</i>	setng	$D \leftarrow (SF \wedge OF) \vee ZF$	小于等于(有符号<=)
seta <i>D</i>	setnbe	$D \leftarrow \neg CF \wedge \neg ZF$	超过(无符号>)
setae <i>D</i>	setnb	$D \leftarrow \neg CF$	超过或相等(无符号>=)
setb <i>D</i>	setnae	$D \leftarrow CF$	低于(无符号<)
setbe <i>D</i>	setna	$D \leftarrow CF \vee ZF$	低于或相等(无符号<=)

图 3-14 SET 指令。每条指令根据条件码的某种组合，将一个字节设置为 0 或者 1。
有些指令有“同义名”，也就是同一条机器指令有别名字

```

int comp(data_t a, data_t b)
a in %rdi, b in %rsi
1  comp:
2      cmpq    %rsi, %rdi    Compare a:b
3      setl    %al          Set low-order byte of %eax to 0 or 1
4      movzbl  %al, %eax     Clear rest of %eax (and rest of %rax)
5      ret

```

注意 `cmpq` 指令的比较顺序(第 2 行)。虽然参数列出的顺序先是 `%rsi(b)` 再是 `%rdi(a)`，实际上比较的是 `a` 和 `b`。还要记得，正如在 3.4.2 节中讨论过的那样，`movzbl` 指令不仅会把 `%eax` 的高 3 个字节清零，还会把整个寄存器 `%rax` 的高 4 个字节都清零。

某些底层的机器指令可能有多个名字，我们称之为“同义名(synonym)”。比如说，`setg`(表示“设置大于”)和 `setnle`(表示“设置不小于等于”)指的就是同一条机器指令。编译器和反汇编器会随意决定使用哪个名字。

虽然所有的算术和逻辑操作都会设置条件码，但是各个 SET 命令的描述都适用的情况是：执行比较指令，根据计算 $t = a - b$ 设置条件码。更具体地说，假设 a 、 b 和 t 分别是变量 a 、 b 和 t 的补码形式表示的整数，因此 $t = a - {}_w b$ ，这里 w 取决于 a 和 b 的大小。

来看 `sete` 的情况，即“当相等时设置(set when equal)”指令。当 $a = b$ 时，会得到 $t = 0$ ，因此零标志位置就表示相等。类似地，考虑用 `setl`，即“当小于时设置(set when less)”指令，测试一个有符号比较。当没有发生溢出时(`OF` 设置为 0 就表明无溢出)，我们有当 $a - {}_w b < 0$ 时 $a < b$ ，将 `SF` 设置为 1 即指明这一点，而当 $a - {}_w b \geq 0$ 时 $a \geq b$ ，由 `SF` 设置为 0 指明。另一方面，当发生溢出时，我们有当 $a - {}_w b > 0$ (负溢出)时 $a < b$ ，而当 $a - {}_w b < 0$ (正溢出)时 $a > b$ 。当 $a = b$ 时，不会有溢出。因此，当 `OF` 被设置为 1 时，当且仅当 `SF` 被设置为 0，有 $a < b$ 。将这些情况组合起来，溢出和符号位的 EXCLUSIVE-OR 提供了 $a < b$ 是否为真的测试。其他的有符号比较测试基于 $SF \wedge OF$ 和 `ZF` 的其他组合。

对于无符号比较的测试，现在设 a 和 b 是变量 a 和 b 的无符号形式表示的整数。在执行计算 $t = a - b$ 中，当 $a - b < 0$ 时，`CMP` 指令会设置进位标志，因而无符号比较使用的是

进位标志和零标志的组合。

注意到机器代码如何区分有符号和无符号值是很重要的。同 C 语言不同，机器代码不会将每个程序值都和一个数据类型联系起来。相反，大多数情况下，机器代码对于有符号和无符号两种情况都使用一样的指令，这是因为许多算术运算对无符号和补码算术都有一样的位级行为。有些情况需要用不同的指令来处理有符号和无符号操作，例如，使用不同版本的右移、除法和乘法指令，以及不同的条件码组合。

 **练习题 3.13** 考虑下列的 C 语言代码：

```
int comp(data_t a, data_t b) {
    return a COMP b;
}
```

它给出了参数 a 和 b 之间比较的一般形式，这里，参数的数据类型 data_t(通过 typedef)被声明为表 3-1 中列出的某种整数类型，可以是有符号的也可以是无符号的 comp 通过 #define 来定义。

假设 a 在 %rdi 中某个部分，b 在 %rsi 中某个部分。对于下面每个指令序列，确定哪种数据类型 data_t 和比较 COMP 会导致编译器产生这样的代码。(可能有多个正确答案，请列出所有的正确答案。)

- A. `cmpl %esi, %edi`
 `setl %al`
- B. `cmpw %si, %di`
 `setge %al`
- C. `cmpb %sil, %dil`
 `setbe %al`
- D. `cmpq %rsi, %rdi`
 `setne %a`

 **练习题 3.14** 考虑下面的 C 语言代码：

```
int test(data_t a) {
    return a TEST 0;
}
```

它给出了参数 a 和 0 之间比较的一般形式，这里，我们可以用 typedef 来声明 data_t，从而设置参数的数据类型，用 #define 来声明 TEST，从而设置比较的类型。对于下面每个指令序列，确定哪种数据类型 data_t 和比较 TEST 会导致编译器产生这样的代码。(可能有多个正确答案，请列出所有的正确答案。)

- A. `testq %rdi, %rdi`
 `setge %al`
- B. `testw %di, %di`
 `sete %al`
- C. `testb %dil, %dil`
 `seta %al`
- D. `testl %edi, %edi`
 `setne %al`

3.6.3 跳转指令

正常执行的情况下，指令按照它们出现的顺序一条一条地执行。跳转(jump)指令会导致执行切换到程序中一个全新的位置。在汇编代码中，这些跳转的目的地通常用一个标号

(label)指明。考虑下面的汇编代码序列(完全是人为编造的):

```
movq $0,%rax           Set %rax to 0
jmp .L1                Goto .L1
movq (%rax),%rdx        Null pointer dereference (skipped)
.L1:
popq %rdx              Jump target
```

指令 `jmp .L1` 会导致程序跳过 `movq` 指令, 而从 `popq` 指令开始继续执行。在产生目标代码文件时, 汇编器会确定所有带标号指令的地址, 并将跳转目标(目的指令的地址)编码为跳转指令的一部分。

图 3-15 列举了不同的跳转指令。`jmp` 指令是无条件跳转。它可以是直接跳转, 即跳转目标是作为指令的一部分编码的; 也可以是间接跳转, 即跳转目标是从寄存器或内存位置中读出的。汇编语言中, 直接跳转是给出一个标号作为跳转目标的, 例如上面所示代码中的标号 “.L1”。间接跳转的写法是 ‘*’ 后面跟一个操作数指示符, 使用图 3-3 中描述的内存操作数格式中的一种。举个例子, 指令

```
jmp *%rax
```

用寄存器 `%rax` 中的值作为跳转目标, 而指令

```
jmp *(%rax)
```

以 `%rax` 中的值作为读地址, 从内存中读出跳转目标。

指令	同义名	跳转条件	描述
<code>jmp Label</code>		1	直接跳转
<code>jmp *Operand</code>		1	间接跳转
<code>jz Label</code>	<code>jz</code>	ZF	相等/零
<code>jne Label</code>	<code>jnz</code>	~ZF	不相等/非零
<code>js Label</code>		SF	负数
<code>jns Label</code>		~SF	非负数
<code>jg Label</code>	<code>jnle</code>	~(SF ^ OF) & ~ZF	大于 (有符号 >)
<code>jge Label</code>	<code>jnl</code>	~(SF ^ OF)	大于或等于 (有符号 >=)
<code>jl Label</code>	<code>jnge</code>	SF ^ OF	小于 (有符号 <)
<code>jle Label</code>	<code>jng</code>	(SF ^ OF) ZF	小于或等于 (有符号 <=)
<code>ja Label</code>	<code>jnb</code>	~CF & ~ZF	超过 (无符号 >)
<code>jae Label</code>	<code>jnb</code>	~CF	超过或相等 (无符号 >=)
<code>jb Label</code>	<code>jnae</code>	CF	低于 (无符号 <)
<code>jbe Label</code>	<code>jna</code>	CF ZF	低于或相等 (无符号 <=)

图 3-15 `jump` 指令。当跳转条件满足时, 这些指令会跳转到一条带标号的目的地。有些指令有“同义名”, 也就是同一条机器指令的别名

表中所示的其他跳转指令都是有条件的——它们根据条件码的某种组合, 或者跳转, 或者继续执行代码序列中下一条指令。这些指令的名字和跳转条件与 `SET` 指令的名字和设置条件是相匹配的(参见图 3-14)。同 `SET` 指令一样, 一些底层的机器指令有多个名字。条件跳转只能是直接跳转。

3.6.4 跳转指令的编码

虽然我们不关心机器代码格式的细节, 但是理解跳转指令的目标如何编码, 这对第 7

章研究链接非常重要。此外，它也能帮助理解反汇编器的输出。在汇编代码中，跳转目标用符号标号书写。汇编器，以及后来的链接器，会产生跳转目标的适当编码。跳转指令有几种不同的编码，但是最常用都是 PC 相对的(PC-relative)。也就是，它们会将目标指令的地址与紧跟在跳转指令后面那条指令的地址之间的差作为编码。这些地址偏移量可以编码为 1、2 或 4 个字节。第二种编码方法是给出“绝对”地址，用 4 个字节直接指定目标。汇编器和链接器会选择适当的跳转目的编码。

下面是一个 PC 相对寻址的例子，这个函数的汇编代码由编译文件 branch.c 产生。它包含两个跳转：第 2 行的 jmp 指令前向跳转到更高的地址，而第 7 行的 jg 指令后向跳转到较低的地址。

```

1    movq    %rdi, %rax
2    jmp     .L2
3    .L3:
4    sarq    %rax
5    .L2:
6    testq   %rax, %rax
7    jg      .L3
8    repz; ret

```

汇编器产生的“.o”格式的反汇编版本如下：

```

1      0:  48 89 f8          mov     %rdi,%rax
2      3:  eb 03          jmp     8 <loop+0x8>
3      5:  48 d1 f8          sar     %rax
4      8:  48 85 c0          test    %rax,%rax
5      b:  7f f8          jg      5 <loop+0x5>
6      d:  f3 c3          repz retq

```

右边反汇编器产生的注释中，第 2 行中跳转指令的跳转目标指明为 0x8，第 5 行中跳转指令的跳转目标是 0x5(反汇编器以十六进制格式给出所有的数字)。不过，观察指令的字节编码，会看到第一条跳转指令的目标编码(在第二个字节中)为 0x03。把它加上 0x5，也就是下一条指令的地址，就得到跳转目标地址 0x8，也就是第 4 行指令的地址。

类似，第二个跳转指令的目标用单字节、补码表示编码为 0xf8(十进制-8)。将这个数加上 0xd(十进制 13)，即第 6 行指令的地址，我们得到 0x5，即第 3 行指令的地址。

这些例子说明，当执行 PC 相对寻址时，程序计数器的值是跳转指令后面的那条指令的地址，而不是跳转指令本身的地址。这种惯例可以追溯到早期的实现，当时的处理器会将更新程序计数器作为执行一条指令的第一步。

下面是链接后的程序反汇编版本：

```

1      4004d0: 48 89 f8          mov     %rdi,%rax
2      4004d3: eb 03          jmp     4004d8 <loop+0x8>
3      4004d5: 48 d1 f8          sar     %rax
4      4004d8: 48 85 c0          test    %rax,%rax
5      4004db: 7f f8          jg      4004d5 <loop+0x5>
6      4004dd: f3 c3          repz retq

```

这些指令被重定位到不同的地址，但是第 2 行和第 5 行中跳转目标的编码并没有变。通过使用与 PC 相对的跳转目标编码，指令编码很简洁(只需要 2 个字节)，而且目标代码可以不做改变就移到内存中不同的位置。

旁注 指令 rep 和 repz 有什么用

本节开始的汇编代码的第8行包含指令组合 rep; ret。它们在反汇编代码中(第6行)对应于 repz retq。可以推测出 repz 是 rep 的同义名,而 retq 是 ret 的同义名。查阅 Intel 和 AMD 有关 rep 的文档,我们发现它通常用来实现重复的字符串操作[3, 51]。在这里用它似乎很不合适。这个问题的答案可以在 AMD 给编译器编写者的指导意见书[1]中找到。他们建议用 rep 后面跟 ret 的组合来避免使 ret 指令成为条件跳转指令的目标。如果没有 rep 指令,当分支不跳转时, jg 指令(汇编代码的第7行)会继续到 ret 指令。根据 AMD 的说法,当 ret 指令通过跳转指令到达时,处理器不能正确预测 ret 指令的目的。这里的 rep 指令就是作为一种空操作,因此作为跳转目的插入它,除了能使代码在 AMD 上运行得更快之外,不会改变代码的其他行为。在本书后面其他代码中再遇到 rep 或 repz 时,我们可以很放心地无视它们。

 **练习题 3.15** 在下面这些反汇编二进制代码节选中,有些信息被 X 代替了。回答下列关于这些指令的问题。

A. 下面 je 指令的目标是什么?(在此,你不需要知道任何有关 callq 指令的信息。)

```
4003fa: 74 02          je      XXXXXX
4003fc: ff d0         callq   %rax
```

B. 下面 je 指令的目标是什么?

```
40042f: 74 f4          je      XXXXXX
400431: 5d             pop     %rbp
```

C. ja 和 pop 指令的地址是多少?

```
XXXXXX: 77 02          ja      400547
XXXXXX: 5d             pop     %rbp
```

D. 在下面的代码中,跳转目标的编码是 PC 相对的,且是一个 4 字节补码数。字节按照从最低位到最高位的顺序列出,反映出 x86-64 的小端法字节顺序。跳转目标的地址是什么?

```
4005e8: e9 73 ff ff ff jmpq    XXXXXXXX
4005ed: 90             nop
```

跳转指令提供了一种实现条件执行(if)和几种不同循环结构的方式。

3.6.5 用条件控制来实现条件分支

将条件表达式和语句从 C 语言翻译成机器代码,最常用的方式是结合有条件和无条件跳转。(另一种方式在 3.6.6 节中会看到,有些条件可以用数据的条件转移实现,而不是用控制的条件转移来实现。)例如,图 3-16a 给出了一个计算两数之差绝对值的函数的 C 代码[⊖]。这个函数有一个副作用,会增加两个计数器,编码为全局变量 lt_cnt 和 ge_cnt 之一。GCC 产生的汇编代码如图 3-16c 所示。把这个机器代码再转换成 C 语言,我们称之为函数 gotodiff_se(图 3-16b)。它使用了 C 语言中的 goto 语句,这个语句类似于汇编代码中的无条件跳转。使用 goto 语句通常认为是一种不好的编程风格,因为它会使代码非

[⊖] 实际上,如果一个减法溢出,这个函数就会返回一个负数值。这里我们主要是为了展示机器代码,而不是实现代码的健壮性。

常难以阅读和调试。本文中使用 goto 语句，是为了构造描述汇编代码程序控制流的 C 程序。我们称这样的编程风格为“goto 代码”。

在 goto 代码中(图 3-16b)，第 5 行中的 goto x_ge_y 语句会导致跳转到第 9 行中的标号 x_ge_y 处(当 $x \geq y$ 时会进行跳转)。从这一点继续执行，完成函数 absdiff_se 的 else 部分并返回。另一方面，如果测试 $x > y$ 失败，程序会计算 absdiff_se 的 if 部分指定的步骤并返回。

汇编代码的实现(图 3-16c)首先比较了两个操作数(第 2 行)，设置条件码。如果比较的结果表明 x 大于或者等于 y ，那么它就会跳转到第 8 行，增加全局变量 ge_cnt，计算 $x - y$ 作为返回值并返回。由此我们可以看到 absdiff_se 对应汇编代码的控制流非常类似于 gotodiff_se 的 goto 代码。

```
long lt_cnt = 0;
long ge_cnt = 0;

long absdiff_se(long x, long y)
{
    long result;
    if (x < y) {
        lt_cnt++;
        result = y - x;
    }
    else {
        ge_cnt++;
        result = x - y;
    }
    return result;
}
```

a) 原始的C语言代码

```
1 long gotodiff_se(long x, long y)
2 {
3     long result;
4     if (x >= y)
5         goto x_ge_y;
6     lt_cnt++;
7     result = y - x;
8     return result;
9 x_ge_y:
10    ge_cnt++;
11    result = x - y;
12    return result;
13 }
```

b) 与之等价的goto版本

```
long absdiff_se(long x, long y)
x in %rdi, y in %rsi
absdiff_se:
1    cmpq    %rsi, %rdi          Compare x:y
2    jge     .L2                 If >= goto x_ge_y
3    addq    $1, lt_cnt(%rip)     lt_cnt++
4    movq    %rsi, %rax
5    subq    %rdi, %rax          result = y - x
6    ret                                Return
7
8 .L2:                          x_ge_y:
9    addq    $1, ge_cnt(%rip)     ge_cnt++
10   movq    %rdi, %rax
11   subq    %rsi, %rax          result = x - y
12   ret                                Return
```

c) 产生的汇编代码

图 3-16 条件语句的编译。a)C 过程 absdiff_se 包含一个 if-else 语句；b)C 过程 gotodiff_se 模拟了汇编代码的控制；c)给出了产生的汇编代码

C语言中的 if-else 语句的通用形式模板如下：

```
if (test-expr)
    then-statement
else
    else-statement
```

这里 *test-expr* 是一个整数表达式，它的取值为 0(解释为“假”)或者为非 0(解释为“真”)。两个分支语句中(*then-statement* 或 *else-statement*)只会执行一个。

对于这种通用形式，汇编实现通常会使用下面这种形式，这里，我们用 C 语法来描述控制流：

```
t = test-expr;
if (!t)
    goto false;
then-statement
goto done;
false:
    else-statement
done:
```

也就是，汇编器为 *then-statement* 和 *else-statement* 产生各自的代码块。它会插入条件和无条件分支，以保证能执行正确的代码块。

旁注 用 C 代码描述机器代码

图 3-16 给出了一个示例，用来展示把 C 语言控制结构翻译成机器代码。图中包括示例的 C 函数 a 和由 GCC 生成的汇编代码的注释版本 c，还有一个与汇编代码结构高度一致的 C 语言版本 b。机器代码的 C 语言表示有助于你理解其中的关键点，能引导你理解实际的汇编代码。

练习题 3.16 已知下列 C 代码：

```
void cond(long a, long *p)
{
    if (p && a > *p)
        *p = a;
}
```

GCC 会产生下面的汇编代码：

```
void cond(long a, long *p)
    a in %rdi, p in %rsi
cond:
    testq    %rsi, %rsi
    je       .L1
    cmpq     %rdi, (%rsi)
    jge      .L1
    movq     %rdi, (%rsi)
.L1:
    rep; ret
```

- A. 按照图 3-16b 中所示的风格，用 C 语言写一个 goto 版本，执行同样的计算，并模拟汇编代码的控制流。像示例中那样给汇编代码加上注解可能会有所帮助。
- B. 请说明为什么 C 语言代码中只有一个 if 语句，而汇编代码包含两个条件分支。

 练习题 3.17 将 if 语句翻译成 goto 代码的另一种可行的规则如下：

```

t = test-expr;
if (t)
    goto true;
else-statement
goto done;
true:
    then-statement
done:

```

A. 基于这种规则，重写 absdiff_se 的 goto 版本。

B. 你能想出选用一种规则而不选用另一种规则的理由吗？

 练习题 3.18 从如下形式的 C 语言代码开始：

```

long test(long x, long y, long z) {
    long val = _____;
    if (_____) {
        if (_____)
            val = _____;
        else
            val = _____;
    } else if (_____)
        val = _____;
    return val;
}

```

GCC 产生如下的汇编代码：

```

long test(long x, long y, long z)
x in %rdi, y in %rsi, z in %rdx
test:
    leaq    (%rdi,%rsi), %rax
    addq    %rdx, %rax
    cmpq    $-3, %rdi
    jge     .L2
    cmpq    %rdx, %rsi
    jge     .L3
    movq    %rdi, %rax
    imulq    %rsi, %rax
    ret
.L3:
    movq    %rsi, %rax
    imulq    %rdx, %rax
    ret
.L2:
    cmpq    $2, %rdi
    jle     .L4
    movq    %rdi, %rax
    imulq    %rdx, %rax
.L4:
    rep; ret

```

填写 C 代码中缺失的表达式。

3.6.6 用条件传送来实现条件分支

实现条件操作的传统方法是通过使用控制的条件转移。当条件满足时，程序沿着一条执行路径执行，而当条件不满足时，就走另一条路径。这种机制简单而通用，但是在现代处理器上，它可能会非常低效。

一种替代的策略是使用数据的条件转移。这种方法计算一个条件操作的两种结果，然后再根据条件是否满足从中选取一个。只有在一些受限制的情况中，这种策略才可行，但是如果可行，就可以用一条简单的条件传送指令来实现它，条件传送指令更符合现代处理器的性能特性。我们将介绍这一策略，以及它在 x86-64 上的实现。

图 3-17a 给出了一个可以用条件传送编译的示例代码。这个函数计算参数 x 和 y 差的绝对值，和前面的例子一样(图 3-16)。不过前面的例子中，分支里有副作用，会修改 `lt_cnt` 或 `ge_cnt` 的值，而这个版本只是简单地计算函数要返回的值。

GCC 为该函数产生的汇编代码如图 3-17c 所示，它与图 3-17b 中所示的 C 函数 `cmovdiff` 有相似的形式。研究这个 C 版本，我们可以看到它既计算了 $y-x$ ，也计算了 $x-y$ ，分别命名为 `rval` 和 `eval`。然后它再测试 x 是否大于等于 y ，如果是，就在函数返回 `rval` 前，将 `eval` 复制到 `rval` 中。图 3-17c 中的汇编代码有相同的逻辑。关键就在于汇编代码的那条 `cmovge` 指令(第 7 行)实现了 `cmovdiff` 的条件赋值(第 8 行)。只有当第 6 行的 `cmpq` 指令表明一个值大于等于另一个值(正如后缀 `ge` 表明的那样)时，才会把数据源寄存器传送到目的。

```
long absdiff(long x, long y)
{
    long result;
    if (x < y)
        result = y - x;
    else
        result = x - y;
    return result;
}
```

a) 原始的C语言代码

```
1  long cmovdiff(long x, long y)
2  {
3      long rval = y-x;
4      long eval = x-y;
5      long ntest = x >= y;
6      /* Line below requires
7         single instruction: */
8      if (ntest) rval = eval;
9      return rval;
10 }
```

b) 使用条件赋值的实现

```
long absdiff(long x, long y)
x in %rdi, y in %rsi
1  absdiff:
2      movq    %rsi, %rax
3      subq    %rdi, %rax      rval = y-x
4      movq    %rdi, %rdx
5      subq    %rsi, %rdx      eval = x-y
6      cmpq    %rsi, %rdi      Compare x:y
7      cmovge  %rdx, %rax      If >=, rval = eval
8      ret                                Return tval
```

c) 产生的汇编代码

图 3-17 使用条件赋值的条件语句的编译。a) C 函数 `absdiff` 包含一个条件表达式；b) C 函数 `cmovdiff` 模拟汇编代码操作；c) 给出产生的汇编代码

为了解为什么基于条件数据传送的代码会比基于条件控制转移的代码(如图 3-16 中那样)性能要好,我们必须了解一些关于现代处理器如何运行的知识。正如我们将在第 4 章和第 5 章中看到的,处理器通过使用流水线(pipelining)来获得高性能,在流水线中,一条指令的处理要经过一系列的阶段,每个阶段执行所需操作的一小部分(例如,从内存取指令、确定指令类型、从内存读数据、执行算术运算、向内存写数据,以及更新程序计数器)。这种方法通过重叠连续指令的步骤来获得高性能,例如,在取一条指令的同时,执行它前面一条指令的算术运算。要做到这一点,要求能够事先确定要执行的指令序列,这样才能保持流水线中充满了待执行的指令。当机器遇到条件跳转(也称为“分支”)时,只有当分支条件求值完成之后,才能决定分支往哪边走。处理器采用非常精密的分支预测逻辑来猜测每条跳转指令是否会执行。只要它的猜测还比较可靠(现代微处理器设计试图达到 90% 以上的成功率),指令流水线中就会充满着指令。另一方面,错误预测一个跳转,要求处理器丢掉它为该跳转指令后所有指令已做的工作,然后再开始用从正确位置处起始的指令去填充流水线。正如我们会看到的,这样一个错误预测会招致很严重的惩罚,浪费大约 15~30 个时钟周期,导致程序性能严重下降。

作为一个示例,我们在 Intel Haswell 处理器上运行 `absdiff` 函数,用两种方法来实现条件操作。在一个典型的应用中, $x < y$ 的结果非常地不可预测,因此即使是最精密的分支预测硬件也只能有大约 50% 的概率猜对。此外,两个代码序列中的计算执行都只需要一个时钟周期。因此,分支预测错误处罚主导着这个函数的性能。对于包含条件跳转的 x86-64 代码,我们发现当分支行为模式很容易预测时,每次调用函数需要大约 8 个时钟周期;而分支行为模式是随机的时候,每次调用需要大约 17.50 个时钟周期。由此我们可以推断出分支预测错误的处罚是大约 19 个时钟周期。这就意味着函数需要的时间范围大约在 8 到 27 个周期之间,这依赖于分支预测是否正确。

旁注 如何确定分支预测错误的处罚

假设预测错误的概率是 p , 如果没有预测错误,执行代码的时间是 T_{OK} , 而预测错误的处罚是 T_{MP} 。那么,作为 p 的一个函数,执行代码的平均时间是 $T_{avg}(p) = (1-p)T_{OK} + p(T_{OK} + T_{MP}) = T_{OK} + pT_{MP}$ 。如果已知 T_{OK} 和 T_{ran} (当 $p=0.5$ 时的平均时间),要确定 T_{MP} 。将参数代入等式,我们有 $T_{ran} = T_{avg}(0.5) = T_{OK} + 0.5T_{MP}$, 所以有 $T_{MP} = 2(T_{ran} - T_{OK})$ 。因此,对于 $T_{OK}=8$ 和 $T_{ran}=17.5$, 我们有 $T_{MP}=19$ 。

另一方面,无论测试的数据是什么,编译出来使用条件传送的代码所需的时间都是大约 8 个时钟周期。控制流不依赖于数据,这使得处理器更容易保持流水线是满的。

 **练习题 3.19** 在一个比较旧的处理器模型上运行,当分支行为模式非常可预测时,我们的代码需要大约 16 个时钟周期,而当模式是随机的时候,需要大约 31 个时钟周期。

A. 预测错误处罚大约是多少?

B. 当分支预测错误时,这个函数需要多少个时钟周期?

图 3-18 列举了 x86-64 上一些可用的条件传送指令。每条指令都有两个操作数:源寄存器或者内存地址 S , 和目的寄存器 R 。与不同的 SET(3.6.2 节)和跳转指令(3.6.3 节)一样,这些指令的结果取决于条件码的值。源值可以从内存或者源寄存器中读取,但是只有在指定的条件满足时,才会被复制到目的寄存器中。

源和目的的值可以是 16 位、32 位或 64 位长。不支持单字节的条件传送。无条件指令的操作数的长度显式地编码在指令名中(例如 `movw` 和 `movl`), 汇编器可以从目标寄存器的名字推断

出条件传送指令的操作数长度，所以对所有的操作数长度，都可以使用同一个的指令名字。

指令	同义名	传送条件	描述
<code>cmovl S, R</code>	<code>cmovz</code>	ZF	相等/零
<code>cmovle S, R</code>	<code>cmovnz</code>	$\sim$ ZF	不相等/非零
<code>cmovs S, R</code>		SF	负数
<code>cmovsl S, R</code>		$\sim$ SF	非负数
<code>cmovg S, R</code>	<code>cmovnle</code>	$\sim$ (SF $\wedge$ OF) & $\sim$ ZF	大于 (有符号>)
<code>cmovge S, R</code>	<code>cmovnl</code>	$\sim$ (SF $\wedge$ OF)	大于或等于 (有符号>=)
<code>cmovl S, R</code>	<code>cmovnge</code>	SF $\wedge$ OF	小于 (有符号<)
<code>cmovle S, R</code>	<code>cmovng</code>	(SF $\wedge$ OF) ZF	小于或等于 (有符号<=)
<code>cmova S, R</code>	<code>cmovnbe</code>	$\sim$ CF & $\sim$ ZF	超过 (无符号>)
<code>cmovae S, R</code>	<code>cmovnb</code>	$\sim$ CF	超过或相等 (无符号>=)
<code>cmovb S, R</code>	<code>cmovnae</code>	CF	低于 (无符号<)
<code>cmovbe S, R</code>	<code>cmovna</code>	CF ZF	低于或相等 (无符号<=)

图 3-18 条件传送指令。当传送条件满足时，指令把源值 S 复制到目的 R。
有些指令是“同义名”，即同一条机器指令的不同名字

同条件跳转不同，处理器无需预测测试的结果就可以执行条件传送。处理器只是读源值(可能是从内存中)，检查条件码，然后要么更新目的寄存器，要么保持不变。我们会在第 4 章中探讨条件传送的实现。

为了理解如何通过条件数据传输来实现条件操作，考虑下面的条件表达式和赋值的通用形式：

```
v = test-expr ? then-expr : else-expr;
```

用条件控制转移的标准方法来编译这个表达式会得到如下形式：

```
if (!test-expr)
    goto false;
v = then-expr;
goto done;
false:
    v = else-expr;
done:
```

这段代码包含两个代码序列：一个对 *then-expr* 求值，另一个对 *else-expr* 求值。条件跳转和无条件跳转结合起来使用是为了保证只有一个序列执行。

基于条件传送的代码，会对 *then-expr* 和 *else-expr* 都求值，最终值的选择基于对 *test-expr* 的求值。可以用下面的抽象代码描述：

```
v = then-expr;
ve = else-expr;
t = test-expr;
if (!t) v = ve;
```

这个序列中的最后一条语句是用条件传送实现的——只有当测试条件 *t* 满足时，*vt* 的值才会被复制到 *v* 中。

不是所有的条件表达式都可以用条件传送来编译。最重要的是，无论测试结果如何，

我们给出的抽象代码会对 *then-expr* 和 *else-expr* 都求值。如果这两个表达式中的任意一个可能产生错误条件或者副作用，就会导致非法的行为。前面的一个例子(图 3-16)就是这种情况。实际上，我们在该例中引入副作用就是为了强制 GCC 用条件转移来实现这个函数。

作为说明，考虑下面这个 C 函数：

```
long cread(long *xp) {
    return (xp ? *xp : 0);
}
```

乍一看，这段代码似乎很适合被编译成使用条件传送，当指针为空时将结果设置为 0，如下面的汇编代码所示：

```
long cread(long *xp)
Invalid implementation of function cread
xp in register %rdi
1  cread:
2      movq    (%rdi), %rax    v = *xp
3      testq   %rdi, %rdi     Test x
4      movl    $0, %edx       Set ve = 0
5      cmovbe  %rdx, %rax     If x==0, v = ve
6      ret                               Return v
```

不过，这个实现是非法的，因为即使当测试为假时，*movq* 指令(第 2 行)对 *xp* 的间接引用还是发生了，导致一个间接引用空指针的错误。所以，必须用分支代码来编译这段代码。

使用条件传送也不总是会提高代码的效率。例如，如果 *then-expr* 或者 *else-expr* 的求值需要大量的计算，那么当相对应的条件不满足时，这些工作就白费了。编译器必须考虑浪费的计算和由于分支预测错误所造成的性能处罚之间的相对性能。说实话，编译器并不具有足够的信息来做出可靠的决定；例如，它们不知道分支会多好地遵循可预测的模式。我们对 GCC 的实验表明，只有当两个表达式都很容易计算时，例如表达式分别都只是一条加法指令，它才会使用条件传送。根据我们的经验，即使许多分支预测错误的开销会超过更复杂的计算，GCC 还是会使用条件控制转移。

所以，总的来说，条件数据传送提供了一种用条件控制转移来实现条件操作的替代策略。它们只能用于非常受限制的情况，但是这些情况还是相当常见的，而且与现代处理器的运行方式更契合。

 **练习题 3.20** 在下面的 C 函数中，我们对 OP 操作的定义是不完整的：

```
#define OP _____ /* Unknown operator */

long arith(long x) {
    return x OP 8;
}
```

当编译时，GCC 会产生如下汇编代码：

```
long arith(long x)
x in %rdi
arith:
    leaq    7(%rdi), %rax
    testq   %rdi, %rdi
    cmovns  %rdi, %rax
    sarq    $3, %rax
    ret
```

- A. OP 进行的是什么操作?
 B. 给代码添加注释, 解释它是如何工作的。

 练习题 3.21 C 代码开始的形式如下:

```
long test(long x, long y) {
    long val = _____;
    if (_____) {
        if (_____)
            val = _____;
        else
            val = _____;
    } else if (_____)
        val = _____;
    return val;
}
```

GCC 会产生如下汇编代码:

```
long test(long x, long y)
x in %rdi, y in %rsi
test:
    leaq    0(,%rdi,8), %rax
    testq   %rsi, %rsi
    jle     .L2
    movq    %rsi, %rax
    subq    %rdi, %rax
    movq    %rdi, %rdx
    andq    %rsi, %rdx
    cmpq    %rsi, %rdi
    cmovge  %rdx, %rax
    ret
.L2:
    addq    %rsi, %rdi
    cmpq    $-2, %rsi
    cmovle  %rdi, %rax
    ret
```

填补 C 代码中缺失的表达式。

3.6.7 循环

C 语言提供了多种循环结构, 即 do-while、while 和 for。汇编中没有相应的指令存在, 可以用条件测试和跳转组合起来实现循环的效果。GCC 和其他汇编器产生的循环代码主要基于两种基本的循环模式。我们会循序渐进地研究循环的翻译, 从 do-while 开始, 然后再研究具有更复杂实现的循环, 并覆盖这两种模式。

1. do-while 循环

do-while 语句的通用形式如下:

```
do
    body-statement
while (test-expr);
```

这个循环的效果就是重复执行 *body-statement*, 对 *test-expr* 求值, 如果求值的结果为非

零，就继续循环。可以看到，*body-statement* 至少会执行一次。

这种通用形式可以被翻译成如下所示的条件和 goto 语句：

```
loop:
    body-statement
    t = test-expr;
    if (t)
        goto loop;
```

也就是说，每次循环，程序会执行循环体里的语句，然后执行测试表达式。如果测试为真，就回去再执行一次循环。

看一个示例，图 3-19a 给出了一个函数的实现，用 do-while 循环来计算函数参数的阶乘，写作 $n!$ 。这个函数只计算 $n > 0$ 时 n 的阶乘的值。

练习题 3.22

- A. 用一个 32 位 int 表示 $n!$ ，最大的 n 的值是多少？
- B. 如果用一个 64 位 long 表示，最大的 n 的值是多少？

图 3-19b 所示的 goto 代码展示了如何把循环变成低级的测试和条件跳转的组合。result 初始化之后，程序开始循环。首先执行循环体，包括更新变量 result 和 n 。然后测试 $n > 1$ ，如果是真，跳转到循环开始处。图 3-19c 所示的汇编代码就是 goto 代码的原型。条件跳转指令 jg(第 7 行)是实现循环的关键指令，它决定了是需要继续重复还是退出循环。

```
long fact_do(long n)
{
    long result = 1;
    do {
        result *= n;
        n = n-1;
    } while (n > 1);
    return result;
}
```

a) C 代码

```
long fact_do_goto(long n)
{
    long result = 1;
loop:
    result *= n;
    n = n-1;
    if (n > 1)
        goto loop;
    return result;
}
```

b) 等价的 goto 版本

```
long fact_do(long n)
n in %rdi
1 fact_do:
2     movl    $1, %eax           Set result = 1
3     .L2:                                loop:
4     imulq   %rdi, %rax         Compute result *= n
5     subq    $1, %rdi           Decrement n
6     cmpq    $1, %rdi           Compare n:1
7     jg      .L2                If >, goto loop
8     rep; ret                   Return
```

c) 对应的汇编代码

图 3-19 阶乘程序的 do-while 版本的代码。条件跳转会使得程序循环

逆向工程像图 3-19c 中那样的汇编代码，需要确定哪个寄存器对应的是哪个程序值。本例中，这个对应关系很容易确定：我们知道 n 在寄存器 `%rdi` 中传递给函数。可以看到寄存器 `%rax` 初始化为 1（第 2 行）。（注意，虽然指令的目的寄存器是 `%eax`，它实际上还会把 `%rax` 的高 4 字节设置为 0。）还可以看到这个寄存器还会在第 4 行被乘法改变值。此外，`%rax` 用来返回函数值，所以通常会用来存放需要返回的程序值。因此我们断定 `%rax` 对应程序值 `result`。

 **练习题 3.23** 已知 C 代码如下：

```
long dw_loop(long x) {
    long y = x*x;
    long *p = &x;
    long n = 2*x;
    do {
        x += y;
        (*p)++;
        n--;
    } while (n > 0);
    return x;
}
```

GCC 产生的汇编代码如下：

```
long dw_loop(long x)
x initially in %rdi
1 dw_loop:
2     movq    %rdi, %rax
3     movq    %rdi, %rcx
4     imulq   %rdi, %rcx
5     leaq    (%rdi,%rdi), %rdx
6     .L2:
7     leaq    1(%rcx,%rax), %rax
8     subq    $1, %rdx
9     testq   %rdx, %rdx
10    jg      .L2
11    rep; ret
```

- A. 哪些寄存器用来存放程序值 x 、 y 和 n ？
- B. 编译器如何消除对指针变量 p 和表达式 $(*p)++$ 隐含的指针间接引用的需求？
- C. 对汇编代码添加一些注释，描述程序的操作，类似于图 3-19c 中所示的那样。

旁注 逆向工程循环

理解产生的汇编代码与原始源代码之间的关系，关键是找到程序值和寄存器之间的映射关系。对于图 3-19 的循环来说，这个任务非常简单，但是对于更复杂的程序来说，就可能是更具挑战性的任务。C 语言编译器常常会重组计算，因此有些 C 代码中的变量在机器代码中没有对应的值；而有时，机器代码中又会引入源代码中不存在的新值。此外，编译器还常常试图将多个程序值映射到一个寄存器上，来最小化寄存器的使用率。

我们描述 `fact_do` 的过程对于逆向工程循环来说，是一个通用的策略。看看在循环之前如何初始化寄存器，在循环中如何更新和测试寄存器，以及在循环之后又如何使用寄存器。这些步骤中的每一步都提供了一个线索，组合起来就可以解开谜团。做好准

备，你会看到令人惊奇的变换，其中有些情况很明显是编译器能够优化代码，而有些情况很难解释编译器为什么要选用那些奇怪的策略。根据我们的经验，GCC 常常做的一些变换，非但不能带来性能好处，反而甚至可能降低代码性能。

2. while 循环

while 语句的通用形式如下：

```
while (test-expr)
    body-statement
```

与 do-while 的不同之处在于，在第一次执行 body-statement 之前，它会对 test-expr 求值，循环有可能就中止了。有很多种方法将 while 循环翻译成机器代码，GCC 在代码生成中使用其中的两种方法。这两种方法使用同样的循环结构，与 do-while 一样，不过它们实现初始测试的方法不同。

第一种翻译方法，我们称之为跳转到中间(jump to middle)，它执行一个无条件跳转到循环结尾处的测试，以此来执行初始的测试。可以用以下模板来表达这种方法，这个模板把通用的 while 循环格式翻译到 goto 代码：

```
goto test;
loop:
    body-statement
test:
    t = test-expr;
    if (t)
        goto loop;
```

作为一个示例，图 3-20a 给出了使用 while 循环的阶乘函数的实现。这个函数能够正确地计算 $0! = 1$ 。它旁边的函数 fact_while_jm_goto(图 3-20b)是 GCC 带优化命令行选项 -Og 时产生的汇编代码的 C 语言翻译。比较 fact_while(图 3-20b)和 fact_do(图 3-19b)的代码，可以看到它们非常相似，区别仅在于循环前的 goto test 语句使得程序在修改 result 或 n 的值之前，先执行对 n 的测试。图的最下面(图 3-20c)给出的是实际产生的汇编代码。

 练习题 3.24 对于如下 C 代码：

```
long loop_while(long a, long b)
{
    long result = _____;
    while (_____) {
        result = _____;
        a = _____;
    }
    return result;
}
```

以命令行选项 -Og 运行 GCC 产生如下代码：

```
long loop_while(long a, long b)
a in %rdi, b in %rsi
1  loop_while:
2      movl    $1, %eax
3      jmp     .L2
```

```

4  .L3:
5      leaq    (%rdi,%rsi), %rdx
6      imulq   %rdx, %rax
7      addq    $1, %rdi
8  .L2:
9      cmpq    %rsi, %rdi
10     jl      .L3
11     rep; ret

```

可以看到编译器使用了跳转到中间的翻译方法，在第3行用 jmp 跳转到以标号 .L2 开始的测试。填写 C 代码中缺失的部分。

```

long fact_while(long n)
{
    long result = 1;
    while (n > 1) {
        result *= n;
        n = n-1;
    }
    return result;
}

```

a) C 代码

```

long fact_while_jm_goto(long n)
{
    long result = 1;
    goto test;
loop:
    result *= n;
    n = n-1;
test:
    if (n > 1)
        goto loop;
    return result;
}

```

b) 等价的 goto 版本

```

long fact_while(long n)
n in %rdi
fact_while:
    movl    $1, %eax      Set result = 1
    jmp     .L5            Goto test
.L6:                                loop:
    imulq   %rdi, %rax     Compute result *= n
    subq    $1, %rdi       Decrement n
.L5:                                test:
    cmpq    $1, %rdi       Compare n:1
    jg      .L6            If >, goto loop
    rep; ret              Return

```

c) 对应的汇编代码

图 3-20 使用跳转到中间翻译方法的阶乘算法的 while 版本的 C 代码和汇编代码。

C 函数 fact_while_jm_goto 说明了汇编代码版本的操作

第二种翻译方法，我们称之为 guarded-do，首先用条件分支，如果初始条件不成立就跳过循环，把代码变换为 do-while 循环。当使用较高优化等级编译时，例如使用命令行选项 -O1，GCC 会采用这种策略。可以用如下模板来表达这种方法，把通用的 while 循环

格式翻译成 do-while 循环:

```
t = test-expr;
if (!t)
    goto done;
do
    body-statement
while (test-expr);
done:
```

相应地, 还可以把它翻译成 goto 代码如下:

```
t = test-expr;
if (!t)
    goto done;
loop:
    body-statement
    t = test-expr;
    if (t)
        goto loop;
done:
```

利用这种实现策略, 编译器常常可以优化初始的测试, 例如认为测试条件总是满足。

再来看个例子, 图 3-21 给出了图 3-20 所示阶乘函数同样的 C 代码, 不过给出的是 GCC 使用命令行选项 -O1 时的编译。图 3-21c 给出实际生成的汇编代码, 图 3-21b 是这个汇编代码更易读的 C 语言表示。根据 goto 代码, 可以看到如果对于 n 的初始值有 $n \leq 1$, 那么将跳过该循环。该循环本身的基本结构与该函数 do-while 版本产生的结构(图 3-19)一样。不过, 一个有趣的特性是, 循环测试(汇编代码的第 9 行)从原始 C 代码的 $n > 1$ 变成了 $n \neq 1$ 。编译器知道只有当 $n > 1$ 时才会进入循环, 所以将 n 减 1 意味着 $n > 1$ 或者 $n = 1$ 。因此, 测试 $n \neq 1$ 就等价于测试 $n \leq 1$ 。

```
long fact_while(long n)
{
    long result = 1;
    while (n > 1) {
        result *= n;
        n = n-1;
    }
    return result;
}
```

a) C 代码

```
long fact_while_gd_goto(long n)
{
    long result = 1;
    if (n <= 1)
        goto done;
loop:
    result *= n;
    n = n-1;
    if (n != 1)
        goto loop;
done:
    return result;
}
```

b) 等价的 goto 版本

图 3-21 使用 guarded-do 翻译方法的阶乘算法的 while 版本的 C 代码和汇编代码。函数 fact_while_gd_goto 说明了汇编代码版本的操作

```

long fact_while(long n)
n in %rdi
1 fact_while:
2     cmpq    $1, %rdi        Compare n:1
3     jle     .L7             If <=, goto done
4     movl    $1, %eax        Set result = 1
5     .L6:                    loop:
6     imulq   %rdi, %rax      Compute result *= n
7     subq    $1, %rdi        Decrement n
8     cmpq    $1, %rdi        Compare n:1
9     jne     .L6             If !=, goto loop
10    rep; ret                Return
11    .L7:                    done:
12    movl    $1, %eax        Compute result = 1
13    ret                        Return

```

c) 对应的汇编代码

图 3-21 (续)

练习题 3.25 对于如下 C 代码：

```

long loop_while2(long a, long b)
{
    long result = _____;
    while (_____) {
        result = _____;
        b = _____;
    }
    return result;
}

```

以命令行选项 -O1 运行 GCC，产生如下代码：

```

a in %rdi, b in %rsi
1 loop_while2:
2     testq   %rsi, %rsi
3     jle     .L8
4     movq    %rsi, %rax
5     .L7:
6     imulq   %rdi, %rax
7     subq    %rdi, %rsi
8     testq   %rsi, %rsi
9     jg      .L7
10    rep; ret
11    .L8:
12    movq    %rsi, %rax
13    ret

```

可以看到编译器使用了 guarded-do 的翻译方法，在第 3 行使用了 jle 指令使得当初测试不成立时，忽略循环代码。填写缺失的 C 代码。注意汇编语言中的控制结构不一定与根据翻译规则直接翻译 C 代码得到的完全一致。特别地，它有两个不同的 ret 指令（第 10 行和第 13 行）。不过，你可以根据等价的汇编代码行为填写 C 代码中缺失的部分。

 练习题 3.26 函数 `fun_a` 有如下整体结构：

```
long fun_a(unsigned long x) {
    long val = 0;
    while ( ... ) {
        :
        :
    }
    return ...;
}
```

GCC C 编译器产生如下汇编代码：

```
long fun_a(unsigned long x)
x in %rdi
1  fun_a:
2      movl    $0, %eax
3      jmp     .L5
4  .L6:
5      xorq    %rdi, %rax
6      shrq    %rdi           Shift right by 1
7  .L5:
8      testq   %rdi, %rdi
9      jne     .L6
10     andl    $1, %eax
11     ret
```

逆向工程这段代码的操作，然后完成下面作业：

- 确定这段代码使用的循环翻译方法。
- 根据汇编代码版本填写 C 代码中缺失的部分。
- 用自然语言描述这个函数是计算什么的。

3. for 循环

for 循环的通用形式如下：

```
for (init-expr; test-expr; update-expr)
    body-statement
```

C 语言标准说明(有一个例外，练习题 3.29 中有特别说明)，这样一个循环的行为与下面这段使用 while 循环的代码的行为一样：

```
init-expr;
while (test-expr) {
    body-statement
    update-expr;
}
```

程序首先对初始表达式 `init-expr` 求值，然后进入循环；在循环中它先对测试条件 `test-expr` 求值，如果测试结果为“假”就会退出，否则执行循环体 `body-statement`；最后对更新表达式 `update-expr` 求值。

GCC 为 for 循环产生的代码是 while 循环的两种翻译之一，这取决于优化的等级。也就是，跳转到中间策略会得到如下 goto 代码：

```
init-expr;
goto test;
```

```

loop:
    body-statement
    update-expr;
test:
    t = test-expr;
    if (t)
        goto loop;

```

而 guarded-do 策略得到:

```

    init-expr;
    t = test-expr;
    if (!t)
        goto done;
loop:
    body-statement
    update-expr;
    t = test-expr;
    if (t)
        goto loop;

```

done:

作为一个示例, 考虑用 for 循环写的阶乘函数:

```

long fact_for(long n)
{
    long i;
    long result = 1;
    for (i = 2; i <= n; i++)
        result *= i;
    return result;
}

```

如上述代码所示, 用 for 循环编写阶乘函数最自然的方式就是将从 2 一直到 n 的因子乘起来, 因此, 这个函数与我们使用 while 或者 do-while 循环的代码很不一样。

这段代码中的 for 循环的不同组成部分如下:

```

init-expr      i = 2
test-expr      i <= n
update-expr    i++
body-statement result *= i;

```

用这些部分替换前面给出的模板中相应的位置, 就把 for 循环转换成了 while 循环, 得到下面的代码:

```

long fact_for_while(long n)
{
    long i = 2;
    long result = 1;
    while (i <= n) {
        result *= i;
        i++;
    }
    return result;
}

```


对 while 循环进行跳转到中间变换，得到如下 goto 代码：

```
long fact_for_jm_goto(long n)
{
    long i = 2;
    long result = 1;
    goto test;
loop:
    result *= i;
    i++;
test:
    if (i <= n)
        goto loop;
    return result;
}
```

确实，仔细查看使用命令行选项 -Og 的 GCC 产生的汇编代码，会发现它非常接近于以下模板：

```
long fact_for(long n)
n in %rdi
fact_for:
    movl    $1, %eax        Set result = 1
    movl    $2, %edx        Set i = 2
    jmp     .L8             Goto test
.L9:                loop:
    imulq   %rdx, %rax       Compute result *= i
    addq    $1, %rdx        Increment i
.L8:                test:
    cmpq    %rdi, %rdx      Compare i:n
    jle     .L9             If <=, goto loop
    rep; ret                Return
```

 **练习题 3.27** 先把 fact_for 转换成 while 循环，再进行 guarded-do 变换，写出 fact_for 的 goto 代码。

综上所述，C 语言中三种形式的所有的循环——do-while、while 和 for——都可以用一种简单的策略来翻译，产生包含一个或多个条件分支的代码。控制的条件转移提供了将循环翻译成机器代码的基本机制。

 **练习题 3.28** 函数 fun_b 有如下整体结构：

```
long fun_b(unsigned long x) {
    long val = 0;
    long i;
    for ( ... ; ... ; ... ) {
        :
        :
    }
    return val;
}
```

GCC C 编译器产生如下汇编代码：

```
long fun_b(unsigned long x)
x in %rdi
1 fun_b:
2     movl    $64, %edx
3     movl    $0, %eax
4     .L10:
5     movq    %rdi, %rcx
```

```

6    andl    $1, %ecx
7    addq    %rax, %rax
8    orq     %rcx, %rax
9    shrq    %rdi           Shift right by 1
10   subq    $1, %rdx
11   jne     .L10
12   rep; ret

```

逆向工程这段代码的操作，然后完成下面的工作：

- 根据汇编代码版本填写 C 代码中缺失的部分。
- 解释循环前为什么没有初始测试也没有初始跳转到循环内部的测试部分。
- 用自然语言描述这个函数是计算什么的。

 **练习题 3.29** 在 C 语言中执行 continue 语句会导致程序跳到当前循环迭代的结尾。

当处理 continue 语句时，将 for 循环翻译成 while 循环的描述规则需要一些改进。例如，考虑下面的代码：

```

/* Example of for loop containing a continue statement */
/* Sum even numbers between 0 and 9 */
long sum = 0;
long i;
for (i = 0; i < 10; i++) {
    if (i & 1)
        continue;
    sum += i;
}

```

- 如果我们简单地直接应用将 for 循环翻译到 while 循环的规则，会得到什么呢？产生的代码会有什么错误呢？
- 如何用 goto 语句来替代 continue 语句，保证 while 循环的行为同 for 循环的行为为完全一样？

3.6.8 switch 语句

switch(开关)语句可以根据一个整数索引值进行多重分支(multiway branching)。在处理具有多种可能结果的测试时，这种语句特别有用。它们不仅提高了 C 代码的可读性，而且通过使用跳转表(jump table)这种数据结构使得实现更加高效。跳转表是一个数组，表项 i 是一个代码段的地址，这个代码段实现当开关索引值等于 i 时程序应该采取的动作。程序代码用开关索引值来执行一个跳转表内的数组引用，确定跳转指令的目标。和使用一组很长的 if-else 语句相比，使用跳转表的优点是执行开关语句的时间与开关情况的数量无关。GCC 根据开关情况的数量和开关情况值的稀疏程度来翻译开关语句。当开关情况数量比较多(例如 4 个以上)，并且值的范围跨度比较小时，就会使用跳转表。

图 3-22a 是一个 C 语言 switch 语句的示例。这个例子有些非常有意思的特征，包括情况标号(case label)跨过一个不连续的区域(对于情况 101 和 105 没有标号)，有些情况有多个标号(情况 104 和 106)，而有些情况则会落入其他情况之中(情况 102)，因为对应该情况的代码段没有以 break 语句结尾。

图 3-23 是编译 switch_eg 时产生的汇编代码。这段代码的行为用 C 语言来描述就是图 3-22b 中的过程 switch_eg_impl。这段代码使用了 GCC 提供的对跳转表的支持，这是

对 C 语言的扩展。数组 `jt` 包含 7 个表项，每个都是一个代码块的地址。这些位置由代码中的标号定义，在 `jt` 的表项中由代码指针指明，由标号加上‘&&’前缀组成。（回想运算符 `&` 创建一个指向数据值的指针。在做这个扩展时，GCC 的作者们创造了一个新的运算符 `&&`，这个运算符创建一个指向代码位置的指针。）建议你研究一下 C 语言过程 `switch_eg_impl`，以及它与汇编代码版本之间的关系。

```
void switch_eg(long x, long n,
               long *dest)
{
    long val = x;

    switch (n) {

    case 100:
        val *= 13;
        break;

    case 102:
        val += 10;
        /* Fall through */

    case 103:
        val += 11;
        break;

    case 104:
    case 106:
        val *= val;
        break;

    default:
        val = 0;
    }

    *dest = val;
}
```

a) switch 语句

```
1  void switch_eg_impl(long x, long n,
2                      long *dest)
3  {
4      /* Table of code pointers */
5      static void *jt[7] = {
6          &&loc_A, &&loc_def, &&loc_B,
7          &&loc_C, &&loc_D, &&loc_def,
8          &&loc_D
9      };
10     unsigned long index = n - 100;
11     long val;
12
13     if (index > 6)
14         goto loc_def;
15     /* Multiway branch */
16     goto *jt[index];
17
18 loc_A:    /* Case 100 */
19     val = x * 13;
20     goto done;
21 loc_B:    /* Case 102 */
22     x = x + 10;
23     /* Fall through */
24 loc_C:    /* Case 103 */
25     val = x + 11;
26     goto done;
27 loc_D:    /* Cases 104, 106 */
28     val = x * x;
29     goto done;
30 loc_def:  /* Default case */
31     val = 0;
32 done:
33     *dest = val;
34 }
```

b) 翻译到扩展的 C 语言

图 3-22 switch 语句示例以及翻译到扩展的 C 语言。该翻译给出了跳转表 `jt` 的结构，以及如何访问它。作为对 C 语言的扩展，GCC 支持这样的表

原始的 C 代码有针对值 100、102-104 和 106 的情况，但是开关变量 `n` 可以是任意整数。编译器首先将 `n` 减去 100，把取值范围移到 0 和 6 之间，创建一个新的程序变量，在我们的 C 版本中称为 `index`。补码表示的负数会映射成无符号表示的大正数，利用这一事实，将 `index` 看作无符号值，从而进一步简化了分支的可能性。因此可以通过测试 `index` 是否大于 6 来判定 `index` 是否在 0~6 的范围之外。在 C 和汇编代码中，根据 `index` 的值，有五个不同的跳转位

置: loc_A(在汇编代码中标识为.L3), loc_B(.L5), loc_C(.L6), loc_D(.L7)和 loc_def(.L8), 最后一个默认的目的地址。每个标号都标识一个实现某个情况分支的代码块。在 C 和汇编代码中, 程序都是将 index 和 6 做比较, 如果大于 6 就跳转到默认的代码处。

```

void switch_eg(long x, long n, long *dest)
x in %rdi, n in %rsi, dest in %rdx
1  switch_eg:
2      subq    $100, %rsi           Compute index = n-100
3      cmpq    $6, %rsi           Compare index:6
4      ja      .L8                 If >, goto loc_def
5      jmp     *.L4(,%rsi,8)       Goto *jt[index]
6  .L3:                                loc_A:
7      leaq    (%rdi,%rdi,2), %rax  3*x
8      leaq    (%rdi,%rax,4), %rdi  val = 13*x
9      jmp     .L2                 Goto done
10 .L5:                                loc_B:
11      addq    $10, %rdi          x = x + 10
12 .L6:                                loc_C:
13      addq    $11, %rdi          val = x + 11
14      jmp     .L2                 Goto done
15 .L7:                                loc_D:
16      imulq   %rdi, %rdi          val = x * x
17      jmp     .L2                 Goto done
18 .L8:                                loc_def:
19      movl    $0, %edi           val = 0
20 .L2:                                done:
21      movq    %rdi, (%rdx)       *dest = val
22      ret                                Return

```

图 3-23 图 3-22 中 switch 语句示例的汇编代码

执行 switch 语句的关键步骤是通过跳转表来访问代码位置。在 C 代码中是第 16 行, 一条 goto 语句引用了跳转表 jt。GCC 支持计算 goto(computed goto), 是对 C 语言的扩展。在我们的汇编代码版本中, 类似的操作是在第 5 行, jmp 指令的操作数有前缀 '*', 表明这是一个间接跳转, 操作数指定一个内存位置, 索引由寄存器 %rsi 给出, 这个寄存器保存着 index 的值。(我们会在 3.8 节中看到如何将数组引用翻译成机器代码。)

C 代码将跳转表声明为一个有 7 个元素的数组, 每个元素都是一个指向代码位置的指针。这些元素跨越 index 的值 0~6, 对应于 n 的值 100~106。可以观察到, 跳转表对重复情况的处理就是简单地对表项 4 和 6 用同样的代码标号(loc_D), 而对于缺失的情况的处理就是对表项 1 和 5 使用默认情况的标号(loc_def)。

在汇编代码中, 跳转表用以下声明表示, 我们添加了一些注释:

```

1      .section      .rodata
2      .align 8      Align address to multiple of 8
3  .L4:
4      .quad .L3      Case 100: loc_A
5      .quad .L8      Case 101: loc_def
6      .quad .L5      Case 102: loc_B
7      .quad .L6      Case 103: loc_C
8      .quad .L7      Case 104: loc_D
9      .quad .L8      Case 105: loc_def
10     .quad .L7      Case 106: loc_D

```


这些声明表明，在叫做“.rodata”（只读数据，Read-Only Data）的目标代码文件的段中，应该有一组 7 个“四”字（8 个字节），每个字的值都是与指定的汇编代码标号（例如.L3）相关联的指令地址。标号.L4 标记出这个分配地址的起始。与这个标号相对应的地址会作为间接跳转（第 5 行）的基地址。

不同的代码块（C 标号 loc_A 到 loc_D 和 loc_def）实现了 switch 语句的不同分支。它们中的大多数只是简单地计算了 val 的值，然后跳转到函数的结尾。类似地，汇编代码块计算了寄存器%rdi 的值，并且跳转到函数结尾处由标号.L2 指示的位置。只有情况标号 102 的代码不是这种模式的，正好说明在原始 C 代码中情况 102 会落到情况 103 中。具体处理如下：以标号.L5 起始的汇编代码块中，在块结尾处没有 jmp 指令，这样代码就会继续执行下一个块。类似地，C 版本 switch_eg_impl 中以标号 loc_B 起始的块的结尾处也没有 goto 语句。

检查所有这些代码需要很仔细的研究，但是关键是领会使用跳转表是一种非常有效的实现多重分支的方法。在我们的例子中，程序可以只用一次跳转表引用就分支到 5 个不同的位置。甚至当 switch 语句有上百种情况的时候，也可以只用一次跳转表访问去处理。

 **练习题 3.30** 下面的 C 函数省略了 switch 语句的主体。在 C 代码中，情况标号是不连续的，而有些情况有多个标号。

```
void switch2(long x, long *dest) {
    long val = 0;
    switch (x) {
        :   Body of switch statement omitted
    }
    *dest = val;
}
```

在编译该函数时，GCC 为程序的初始部分生成了以下汇编代码，变量 x 在寄存器%rdi 中：

```
void switch2(long x, long *dest)
x in %rdi
1  switch2:
2      addq    $1, %rdi
3      cmpq    $8, %rdi
4      ja      .L2
5      jmp     *.L4(,%rdi,8)
```

为跳转表生成以下代码：

```
1  .L4:
2      .quad    .L9
3      .quad    .L5
4      .quad    .L6
5      .quad    .L7
6      .quad    .L2
7      .quad    .L7
8      .quad    .L8
9      .quad    .L2
10     .quad    .L5
```

根据上述信息回答下列问题：

- A. switch 语句内情况标号的值分别是多少？
- B. C 代码中哪些情况有多个标号？

 练习题 3.31 对于一个通用结构的 C 函数 switcher:

```

void switcher(long a, long b, long c, long *dest)
{
    long val;
    switch(a) {
        case _____:      /* Case A */
            c = _____;
            /* Fall through */
        case _____:      /* Case B */
            val = _____;
            break;
        case _____:      /* Case C */
        case _____:      /* Case D */
            val = _____;
            break;
        case _____:      /* Case E */
            val = _____;
            break;
        default:
            val = _____;
    }
    *dest = val;
}

```

GCC 产生如图 3-24 所示的汇编代码和跳转表。

```

void switcher(long a, long b, long c, long *dest)
a in %rdi, b in %rsi, c in %rdx, dest in %rcx
1  switcher:
2      cmpq    $7, %rdi
3      ja     .L2
4      jmp     *.L4(,%rdi,8)
5      .section      .rodata
6      .L7:
7      xorq    $15, %rsi
8      movq    %rsi, %rdx
9      .L3:
10     leaq    112(%rdx), %rdi
11     jmp     .L6
12     .L5:
13     leaq    (%rdx,%rsi), %rdi
14     salq    $2, %rdi
15     jmp     .L6
16     .L2:
17     movq    %rsi, %rdi
18     .L6:
19     movq    %rdi, (%rcx)
20     ret

```

a) 代码

```

1      .L4:
2      .quad   .L3
3      .quad   .L2
4      .quad   .L5
5      .quad   .L2
6      .quad   .L6
7      .quad   .L7
8      .quad   .L2
9      .quad   .L5

```

b) 跳转表

图 3-24 练习题 3.31 的汇编代码和跳转表

填写 C 代码中缺失的部分。除了情况标号 C 和 D 的顺序之外，将不同情况填入这个模板的方式是唯一的。

3.7 过程

过程是软件中一种很重要的抽象。它提供了一种封装代码的方式，用一组指定的参数和一个可选的返回值实现了某种功能。然后，可以在程序中不同的地方调用这个函数。设计良好的软件用过程作为抽象机制，隐藏某个行为的具体实现，同时又提供清晰简洁的接口定义，说明要计算的是哪些值，过程会对程序状态产生什么样的影响。不同编程语言中，过程的形式多样：函数(function)、方法(method)、子例程(subroutine)、处理函数(handler)等等，但是它们有一些共有的特性。

要提供对过程的机器级支持，必须要处理许多不同的属性。为了讨论方便，假设过程 P 调用过程 Q，Q 执行后返回到 P。这些动作包括下面一个或多个机制：

传递控制。在进入过程 Q 的时候，程序计数器必须被设置为 Q 的代码的起始地址，然后在返回时，要把程序计数器设置为 P 中调用 Q 后面那条指令的地址。

传递数据。P 必须能够向 Q 提供一个或多个参数，Q 必须能够向 P 返回一个值。

分配和释放内存。在开始时，Q 可能需要为局部变量分配空间，而在返回前，又必须释放这些存储空间。

x86-64 的过程实现包括一组特殊的指令和一些对机器资源(例如寄存器和程序内存)使用的约定规则。人们花了大量的力气来尽量减少过程调用的开销。所以，它遵循了被认为是最低要求策略的方法，只实现上述机制中每个过程所必需的那些。接下来，我们一步步地构建起不同的机制，先描述控制，再描述数据传递，最后是内存管理。

3.7.1 运行时栈

C 语言过程调用机制的一个关键特性(大多数其他语言也是如此)在于使用了栈数据结构提供的后进先出的内存管理原则。在过程 P 调用过程 Q 的例子中，可以看到当 Q 在执行时，P 以及所有在向上追溯到 P 的调用链中的过程，都是暂时被挂起的。当 Q 运行时，它只需要为局部变量分配新的存储空间，或者设置到另一个过程的调用。另一方面，当 Q 返回时，任何它所分配的局部存储空间都可以被释放。因此，程序可以用栈来管理它的过程所需要的存储空间，栈和程序寄存器存放着传递控制和数据、分配内存所需要的信息。当 P 调用 Q 时，控制和数据信息添加到栈尾。当 P 返回时，这些信息会释放掉。

如 3.4.4 节中讲过的，x86-64 的栈向低地址方向增长，而栈指针 `%rsp` 指向栈顶元素。可以用 `pushq` 和 `popq` 指令将数据存入栈中或是从栈中取出。将栈指针减小一个适当的量可以为没有指定初始值的数据在栈上分配空间。类似地，可以通过增加栈指针来释放空间。

当 x86-64 过程需要的存储空间超出寄存器能够存放的大小时，就会在栈上分配空间。这个部分称为过程的栈帧(stack fram)。图 3-25

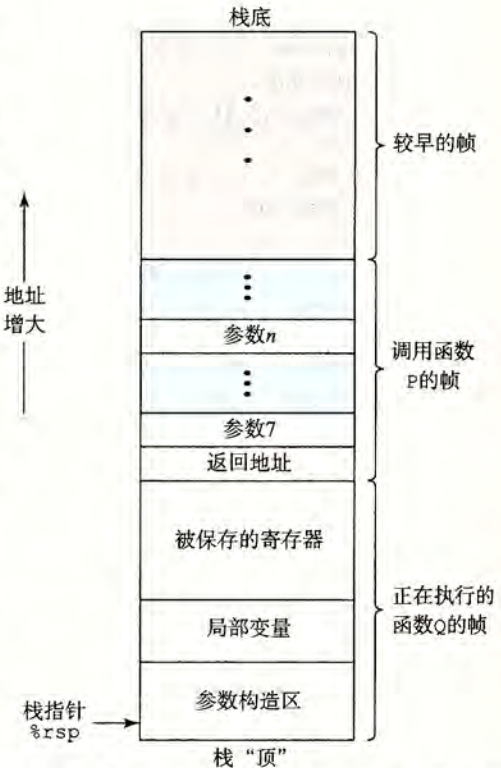


图 3-25 通用的栈帧结构(栈用来传递参数、存储返回信息、保存寄存器，以及局部存储。省略了不必要的部分)